Projet Cloud

Objectif

Déployer une application web sur AWS avec :

- 4 services cloud

- 1 service serverless

- application accessible en ligne

Architecture

- EC2 : hébergement application Docker

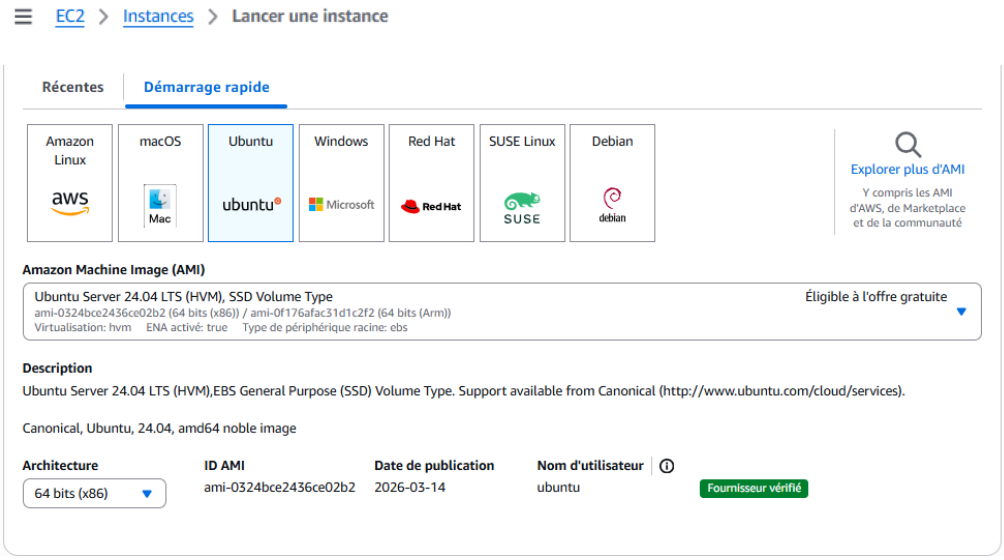
- S3 : stockage backups PostgreSQL

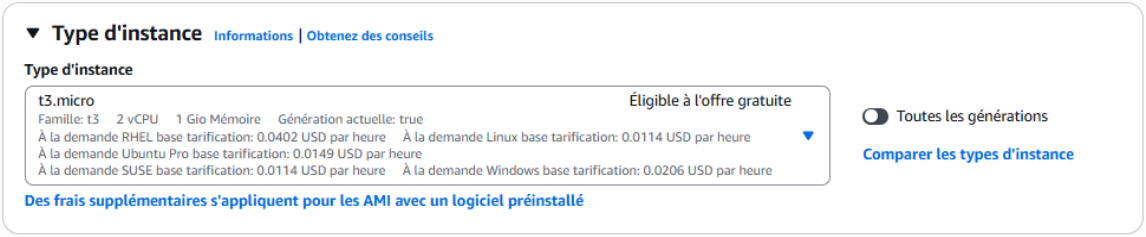
- Lambda : déclenche backup

- EventBridge Scheduler : automatisation

Déploiement EC2

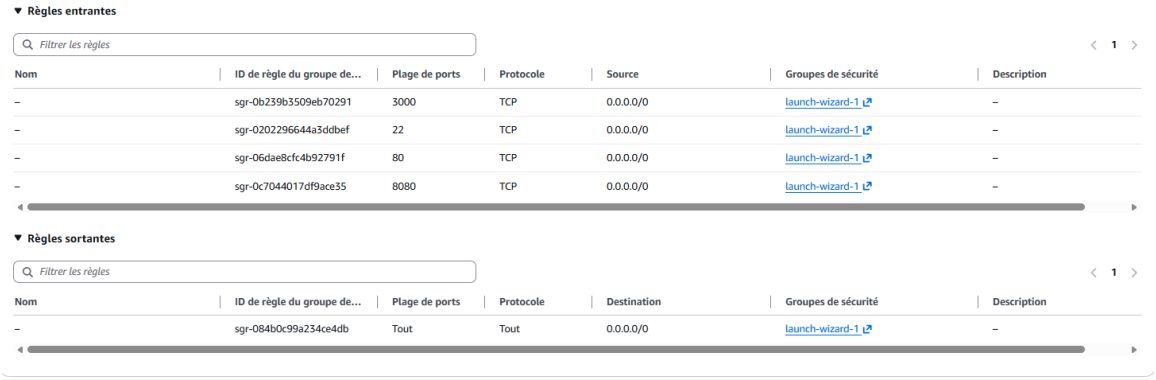
Créer instance Ubuntu



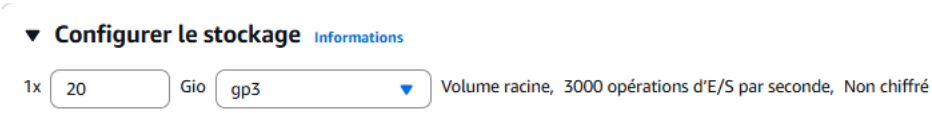


Ouvrir ports : 22, 80, 3000, 8080

Création du groupe de sécurité : launch-wizard-1



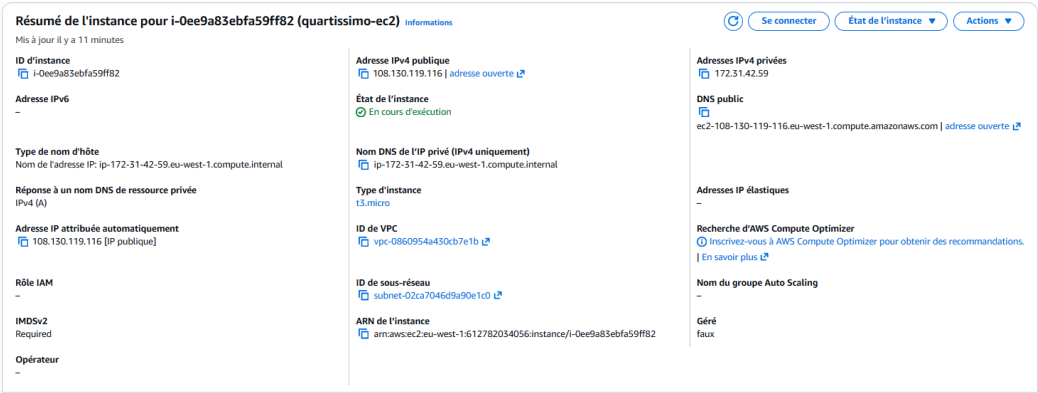
Stockage : 20 GO



Nom du vpc : quartissimo-ec2

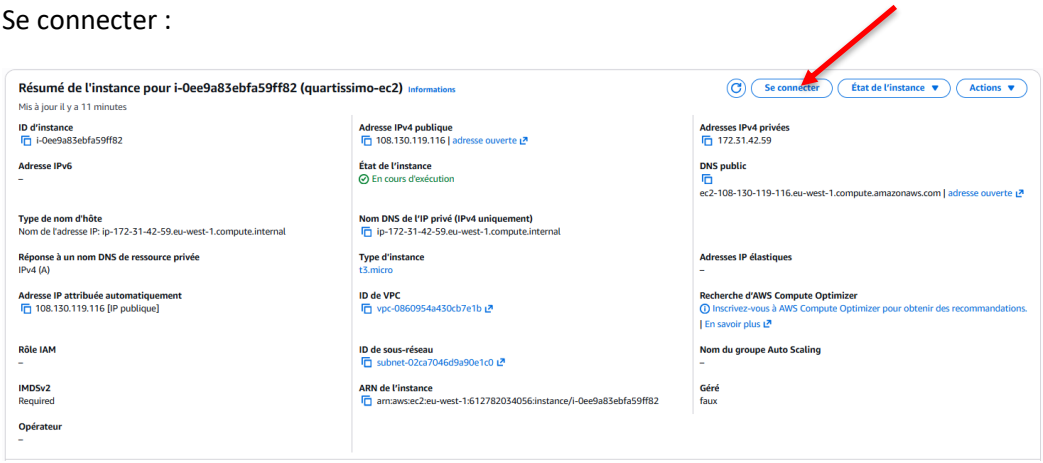
L'instance est maintenant créée.

Résumé de l'instance :



L’IPv4 du VPC change à chaque redémarrage.

Se connecter :



Une fois connecter au vpc, on peut installer Docker pour pouvoir lancer l’application :

[Ubuntu | Docker Docs]

Une fois Docker installer, on peut lancer le projet :

```
git clone https://github.com/bori-to/cloud_projet.git cloud
```

Modifier le fichier frontend/script.js :

const API = "http://IPV4_VPC:3000";

On peut lancer le projet avec :

```
docker compose up --build
```

Utilisation :

Front : http://IPV4_VPC
API:

- http://IPV4_VPC:3000/messages
- POST IPV4_VPC :3000/backups (pas encore fonctionnel)

S3

Création :

- Nom : quartissimo-612782034056-eu-west-1-an

- Région : eu-west-1

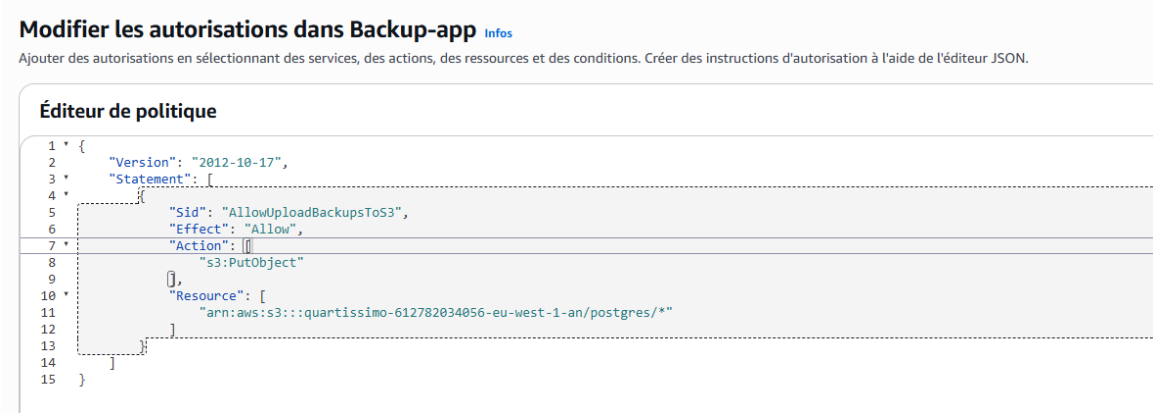
- Versioning activé

Maintenant il faut donner les droits au backend de l'app pour pouvoir effectuer une sauvegarde de la bdd dans le S3.

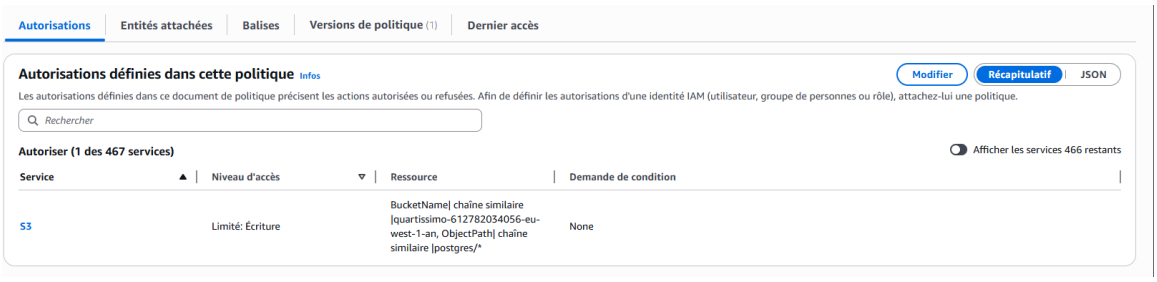
On va créer un utilisateur IAM qui auras les droits d'écriture dans le S3.

Tout d'abord on va créer une politique :

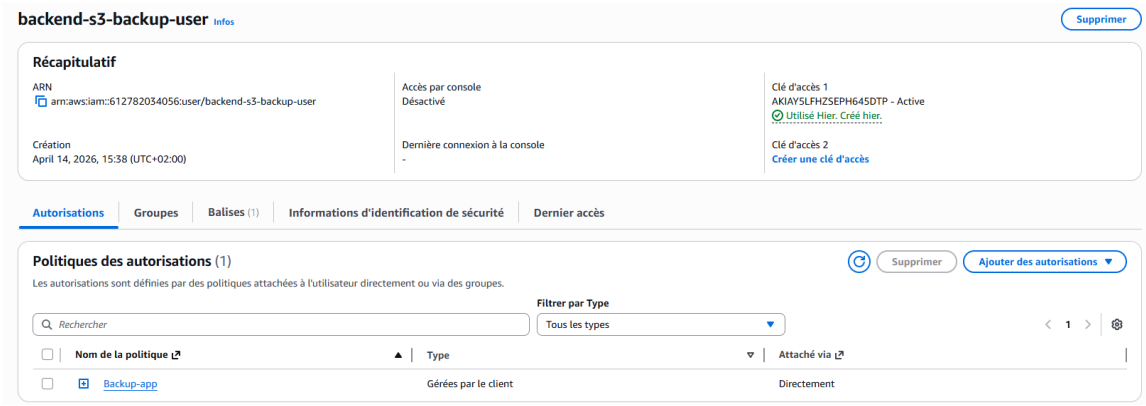
Non : Backup-app



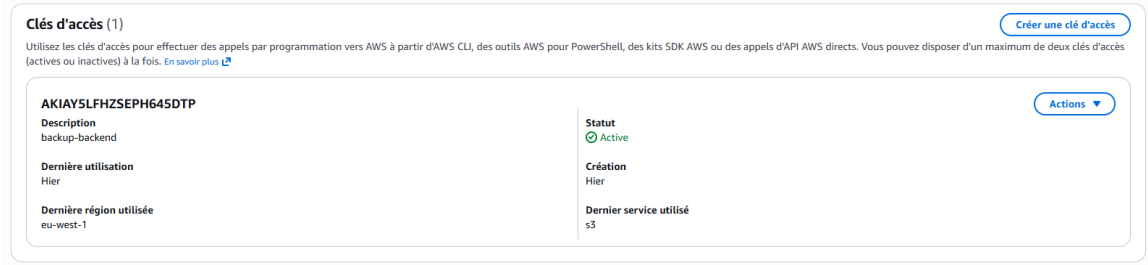
Ici on met les droits en Ecriture dans le dossier postgres/ sur le s3 précédemment créer.



Ensuite on va créer un user et on va lui rattacher cette politique.



Une fois l'user créer, on va lui créer une clé d'accès :



Une fois la clé d'accès récupérée, on va modifier les variables dans le docker-compose.yaml du projet :

BACKUP_SECRET: backups

AWS_REGION: eu-west-1

AWS_ACCESS_KEY_ID: ACCESS_KEY

AWS_SECRET_ACCESS_KEY: SECRET_KEY

S3_BUCKET_NAME: quartissimo-612782034056-eu-west-1-an

S3_BACKUP_PREFIX: postgres

On peut maintenant redémarrer notre app :

```
docker compose up --build
```

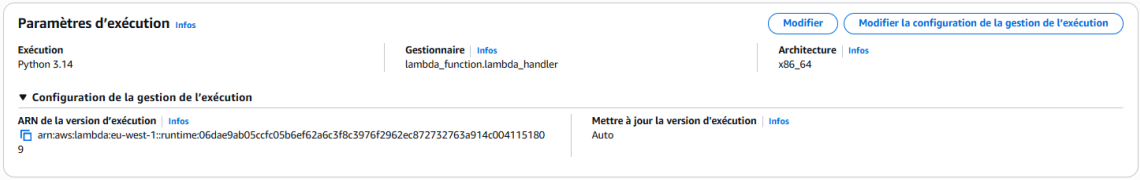
Maintenant on peut faire un backup de la bdd :

```
curl -X POST http://IPV4_VPC:3000/backups -H "x-backup-secret: backups"
```

Lambda

Création du Lambda qui va faire appel à la route api pour faire la save

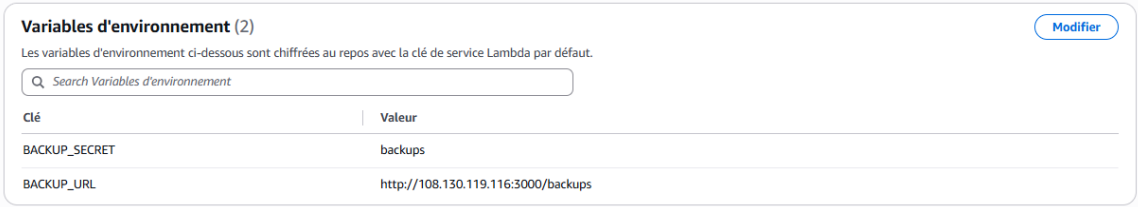
Name : quartissimo-backup



Variables :

BACKUP_URL=http://IPV4_VPC:3000/backups

BACKUP_SECRET=backups

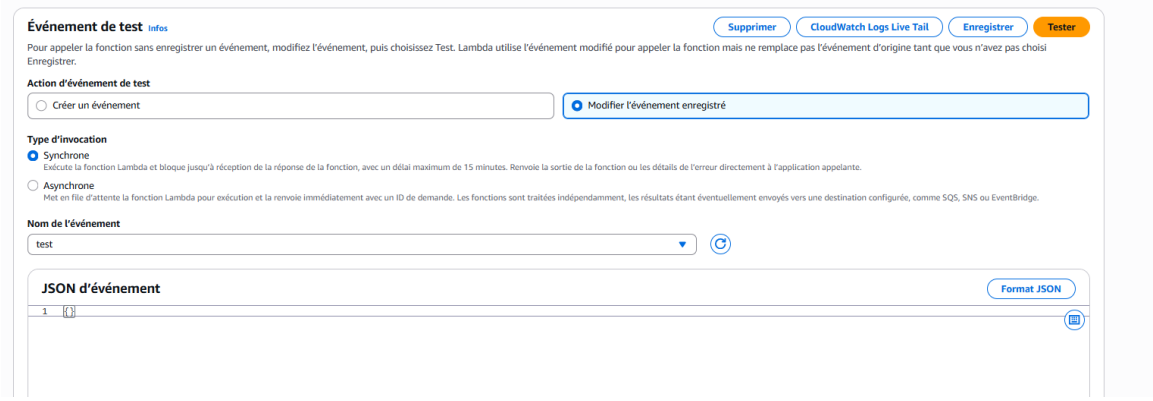


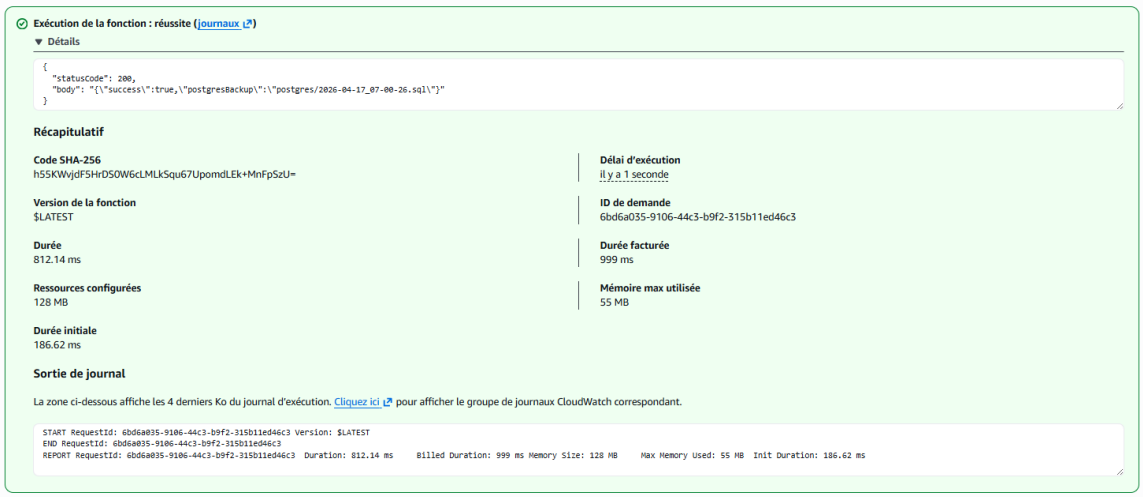
Code :

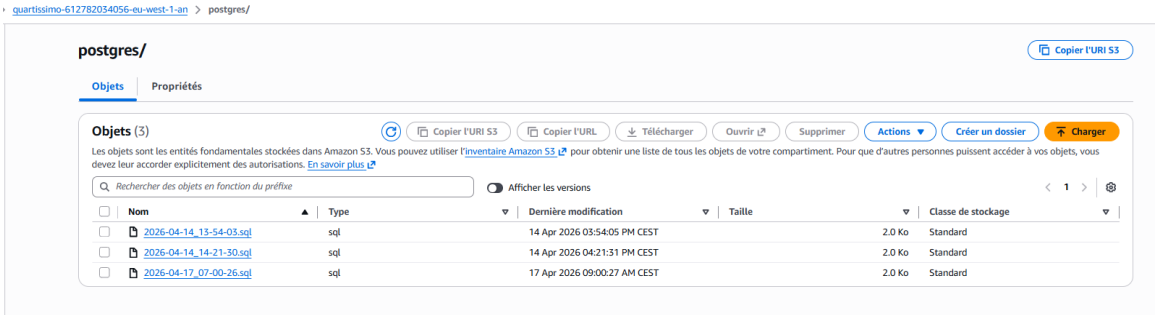
```
import os
import json
import urllib.request
import urllib.error

def lambda_handler(event, context):
    url = os.environ["BACKUP_URL"]
    secret = os.environ["BACKUP_SECRET"]
```

```
    req = urllib.request.Request(
        url=url,
        method="POST",
        headers={
            "x-backup-secret": secret,
            "Content-Type": "application/json"
        }
    )

    try:
        with urllib.request.urlopen(req, timeout=20) as response:
            body = response.read().decode("utf-8")
            return {
                "statusCode": response.status,
                "body": body
            }

    except urllib.error.HTTPError as e:
        error_body = e.read().decode("utf-8", errors="replace")
        return {
            "statusCode": e.code,
            "body": json.dumps({
                "error": "HTTPError",
                "details": error_body
            })
        }

    except Exception as e:
        return {
            "statusCode": 500,
            "body": json.dumps({
                "error": "Exception",
                "details": str(e)
            })
        }
```

Test du Lambda :





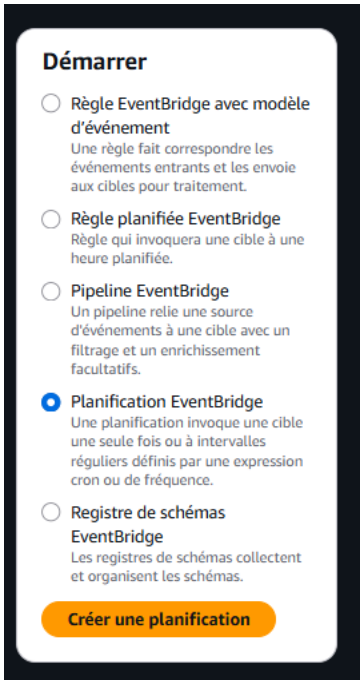


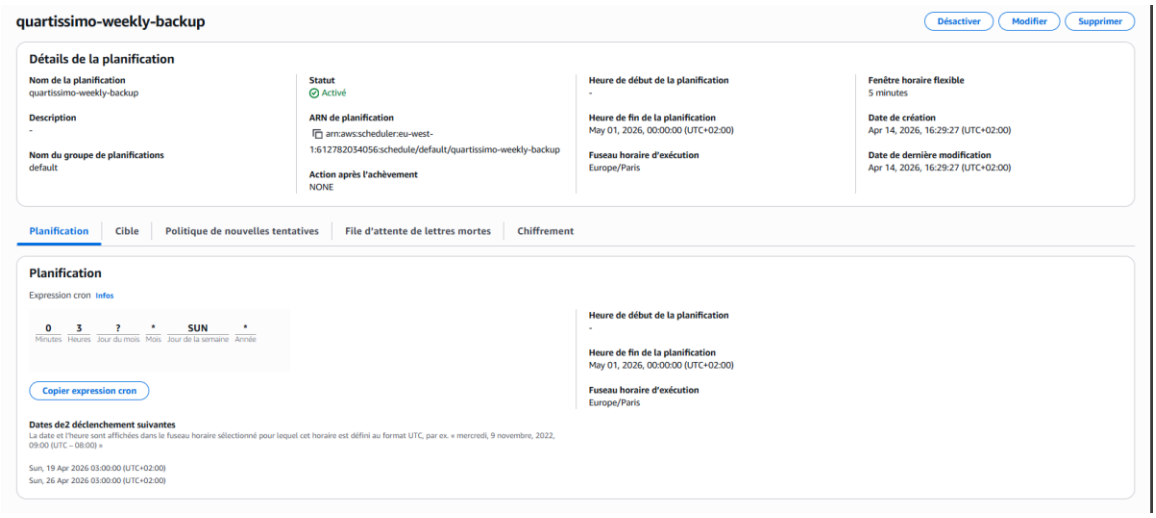
On peut voir dans le S3 que la save a bien fonctionné.

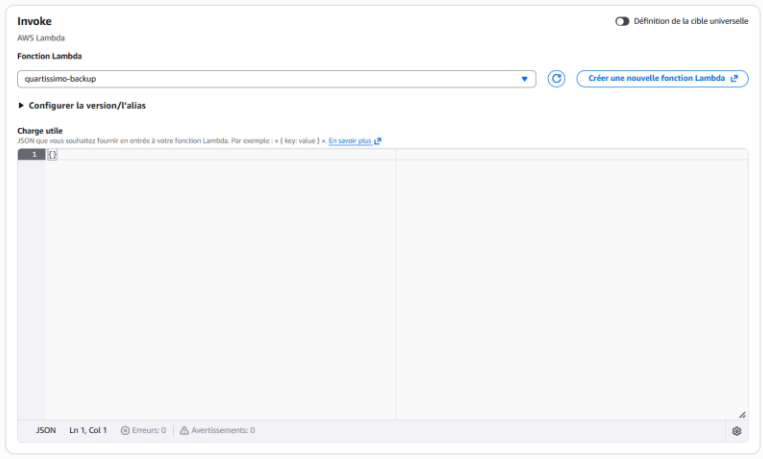
EventBridge

Ce service va permettre de planifier les exécutions du Lambda.

Amazon EventBridge


Démarrer
Règle EventBridge avec modèle d'événement
Une règle fait correspondre les événements entrants et les envoie aux cibles pour traitement.
Règle planifiée EventBridge
Règle qui invoquera une cible à une heure planifiée.
Pipeline EventBridge
Un pipeline relie une source d'événements à une cible avec un filtrage et un enrichissement facultatifs.
Planification EventBridge
Une planification invoque une cible une seule fois ou à intervalles réguliers définis par une expression cron ou de fréquence.
Registre de schémas EventBridge
Les registres de schémas collectent et organisent les schémas.
Créer une planification


quartissimo-weekly-backup
Désactiver
Modifier
Supprimer
Détails de la planification
Nom de la planification
quartissimo-weekly-backup
Description
-
Nom du groupe de planifications
default
Statut
Activé
ARN de planification
arn:aws:scheduler:eu-west-1:612782034056:schedule/default/quartissimo-weekly-backup
Action après l'achèvement
NONE
Heure de début de la planification
-
Heure de fin de la planification
May 01, 2026, 00:00:00 (UTC+02:00)
Fuseau horaire d'exécution
Europe/Paris
Fenêtre horaire flexible
5 minutes
Date de création
Apr 14, 2026, 16:29:27 (UTC+02:00)
Date de dernière modification
Apr 14, 2026, 16:29:27 (UTC+02:00)
Planification
Cible
Politique de nouvelles tentatives
File d'attente de lettres mortes
Chiffrement
Planification
Expression cron Infos
0 3 ? * SUN *
Minutes Heures Jour du mois Mois Jour de la semaine Année
Copier expression cron
Heure de début de la planification
-
Heure de fin de la planification
May 01, 2026, 00:00:00 (UTC+02:00)
Fuseau horaire d'exécution
Europe/Paris
Dates de2 déclenchement suivantes
La date et l'heure sont affichées dans le fuseau horaire sélectionné pour lequel cet horaire est défini au format UTC, par ex. « mercredi, 9 novembre, 2022, 09:00 (UTC – 08:00) »
Sun, 19 Apr 2026 03:00:00 (UTC+02:00)
Sun, 26 Apr 2026 03:00:00 (UTC+02:00)


Invoke
AWS Lambda
Définition de la cible universelle
Fonction Lambda
quartissimo-backup
Créer une nouvelle fonction Lambda
Configurer la version/l'alias
Charge utile
JSON que vous souhaitez fournir en entrée à votre fonction Lambda. Par exemple : « { key: value } ». En savoir plus
JSON
Ln 1, Col 1
Erreurs: 0
Avertissements: 0

Toute les semaines le dimanche à 03 :00

cron(0 3 ? * SUN *)

Ou rate(5 minutes)

Europe/Paris